

Trabajo Final 2026 Integrador

Diseño y Módulos

Grupo 152:

Tiziano Caamaño, Fabián Cardozo, Lautaro Cejas

Tutor:

Sergio Andres Antonini

27 / 09

Índice

Introducción y Objetivos de la Segunda Entrega	2
Arquitectura y Modularización del Sistema	3
Arquitectura en Capas (Presentación, Aplicación, Dominio e Infraestructura)	3
Capa de Presentación (Presentation Layer)	3
Capa de Aplicación (Application Layer)	4
Capa de Dominio (Domain Layer)	4
Capa de Infraestructura (Infrastructure Layer)	4
Diagrama de Paquetes UML	6
Listado y Responsabilidad de los Módulos del Repositorio	7
Diseño y Esquema de la Base de Datos	8
Justificación del Enfoque No Relacional (MongoDB)	9
Diagrama Conceptual y Estructura Documental	9
Estrategia de Almacenamiento: Referencias y Documentos Embebidos	9
Anexo	10



Introducción y Objetivos de la Segunda Entrega

El presente documento constituye la Segunda Entrega (denominada "Diseño y Módulos") del Trabajo Final Integrador, cuyo propósito es establecer los cimientos técnicos e ingenieriles de **MeVe - Marketplace Hiperlocal de Cercanía**. En la primera entrega se identificó el problema: los micro-comerciantes del barrio venden de manera improvisada a través de whatsapp, sin un catálogo ordenado, sin un control de stock y sin una forma de organizar cuando pasa cada comprador a retirar su pedido. Esta etapa traduce esos requerimientos de negocio en un diseño técnico concreto: la arquitectura del sistema, sus módulos y esquema de la base de datos.

Meve es una plataforma de comercio O2O(Online-to-Offline): el comprador arma su pedido en la vitrina digital, lo paga a través de Mercado Pago y lo retira en persona dentro de un franja horaria asignada por el **Gestor de Retiros**. Este gestor evita que un comerciante reciba más retiros de los que puede atender y que venda stock ya comprometido en otro pedido. Coordinar catálogos heterogéneos, cobros de un tercero y reservas con vencimiento exige que cada parte del sistema tenga límites claros.

Por este motivo, el diseño se organiza bajo el principio de **Separación de Responsabilidades(SoC - Separation of Concerns)**, aplicando los dos criterios que plantea la cátedra: **alta cohesión**, es decir, cada módulo con una única responsabilidad clara, y **bajo acoplamiento**, es decir, solo las dependencias imprescindibles y siempre en una misma dirección. Las estructuras se representan mediante un Diagrama de paquetes UML elaborado en UMLetino.

los objetivos de esta entrega son:

- Definir la arquitectura en capas del sistema y la responsabilidad de cada capa.
- Identificar los módulos que componen cada capa y las dependencias entre ellos, representados en un Diagrama de Paquetes UML.
- Relacionar cada módulo lógico con su ubicación dentro del repositorio del proyecto.
- Diseñar el esquema documental de la base de datos en MongoDB y justificar la estrategia de almacenamiento.

Arquitectura y Modularización del Sistema

Arquitectura en Capas (Presentación, Aplicación, Dominio e Infraestructura)

Para cumplir con los principios de **alta cohesión** y **bajo acoplamiento**, MeVe se organiza en cuatro capas lógicas: Presentación, Aplicación, Dominio e Infraestructura. Cada capa agrupa módulos con un mismo tiempo de responsabilidad y solo puede depender de la capa inmediata inferior, sin dependencias circulares saltos de capa.

La única excepción deliberada es la capa de Infraestructura. En lugar de que el dominio depende de MongoDB o de MercadoPago, las capas de Dominio y Aplicación define **interfaces(puertos)**, como RepoProductos o PasarelaDePago, ya la Capa de Infraestructura las implementa. Este mecanismo se conoce como **inversión de dependencias**: las dependencias de infraestructuras apuntan hacia las capas superiores, y la lógica de negocio queda aislada de las tecnologías concretas. Gracias a esto, es posible cambiar la forma de persistir los datos o de procesar los cobros sin modificar las reglas del Gestor de Retiros.

Capa de Presentación (Presentation Layer)

Esta capa representa la frontera del sistema. Es la única responsable de interactuar con el mundo exterior, ya sea recibiendo las acciones del usuario final o procesando las peticiones HTTP entrantes. Su responsabilidad es capturar la intención del usuario, validar el formato de entrada y delegar la ejecución a la capa subyacente, para luego devolver una respuesta estructurada. En MeVe, esta capa se materializa en dos frentes coordinados:

- **Módulo de Interfaz de Usuario (Cliente Web):** Desarrollado en React y TypeScript, abarca la "Vitrina Digital" (vista del comprador) y el panel de autogestión (vista del vendedor). Este módulo tiene una cohesión centrada en la **experiencia de usuario (UX) y la accesibilidad extrema**. Encapsula la lógica de renderizado condicional, la gestión del estado global del *frontend* y la aplicación de parámetros inclusivos (modo oscuro, filtros de daltonismo, ajustes tipográficos), garantizando que los vendedores no técnicos interactúen con el sistema sin fricción.
- **Módulo de Controladores (API REST):** Desarrollado en Spring Boot, contiene los controladores (ej. `OrderController`, `ProductController`, `AuthController`). Su única responsabilidad es exponer los *endpoints*, aplicar los filtros de seguridad (desencriptando el JWT) y transformar los objetos JSON entrantes (DTOs) para pasarlos a la Capa de Aplicación.
- **Módulo Receptor de Webhooks:** Contiene el `WebhookController`, que recibe las notificaciones de pago enviadas por MercadoPago. Se ubica en esta capa porque constituye un punto de entrada HTTP al sistema, al igual que cualquier otro controlador; su única tarea es validar la notificación y delegarla en el Orquestador de Transacciones.

Capa de Aplicación (Application Layer)

También conocida como capa de Casos de Uso. Esta capa no contiene reglas de negocio puras ni detalles de acceso a datos, sino que funciona como el **orquestador central** de MeVe. Su misión es coordinar los distintos componentes del Dominio y de la Infraestructura para cumplir con un flujo específico solicitado por el usuario.

- **Módulo de Autenticación:** Resuelve el registro y el inicio de sesión de los vendedores y la emisión del token de sesión, apoyándose en las reglas de Gestión Multitenant.
- **Módulo de Gestión de Catálogo:** Coordina el alta, la baja y la modificación de productos por parte del vendedor, siempre dentro de su propio comercio, y la consulta de la vitrina por parte del comprador.
- **Módulo Orquestador de Transacciones (Checkout):** Organiza el flujo crítico del O2O Commerce. Cuando un comprador arma su carrito, este módulo coordina el llamado al Dominio para validar la reserva del turno, luego llama a la Infraestructura para iniciar la orden de cobro en MercadoPago, y finalmente gestiona la respuesta al Controlador.
- **Módulo de Tareas Programadas y Eventos (Schedulers):** Encapsula la lógica de procesos automáticos o asíncronos. Aquí reside el coordinador del *rollback* (mediante `@Scheduled` en Spring Boot), que se encarga de monitorear las reservas en estado `PENDIENTE_PAGO` y coordinar la liberación de la franja horaria y la devolución del stock si transcurren 15 minutos sin confirmación del webhook.

Capa de Dominio (Domain Layer)

Representa el corazón del sistema. En esta capa reside la **lógica de negocio pura**, el comportamiento y el estado de los conceptos fundamentales de MeVe. Esta capa es completamente agnóstica de las tecnologías externas: no sabe que existe MongoDB, ni Spring Boot, ni React. Al no tener dependencias hacia ninguna otra capa (acoplamiento nulo hacia el exterior), se vuelve altamente testeable y mantenible.

- **Módulo Gestor de Retiros (Reservas):** Contiene las reglas críticas que dan valor agregado al proyecto. Evalúa la disponibilidad de franjas horarias, calcula si se alcanzó el límite de capacidad de un comerciante (ej. máximo 3 turnos por hora) y determina si un cupo puede pasar a estado de "reserva preventiva" o si debe bloquearse.
- **Módulo de Catálogo e Inventario:** Define la entidad Producto y las reglas de consistencia del stock: no vender más unidades de las disponibles, aplicar la baja lógica mediante el atributo `activo` y no operar sobre productos de otro comercio. Define el puerto **RepoProductos**.
- **Módulo de Gestión Multitenant (Usuarios y Roles):** Modela el Comercio (tenant), el Usuario y sus roles (Vendedor y SuperAdmin), y establece las reglas de pertenencia y autorización entre ellos. El comprador no es un usuario registrado: sus datos de contacto se almacenan embebidos en el pedido. Define los puertos **RepoComercios**, **RepoUsuarios** y **ServicioTokens**.

Capa de Infraestructura (Infrastructure Layer)

Esta es la capa más externa en el lado del servidor, encargada de materializar los detalles técnicos, la persistencia y la comunicación con servicios de terceros. Implementa las interfaces (contratos) definidas por las capas de Aplicación y Dominio, asegurando que el corazón del sistema no se contamine con código de integraciones externas.

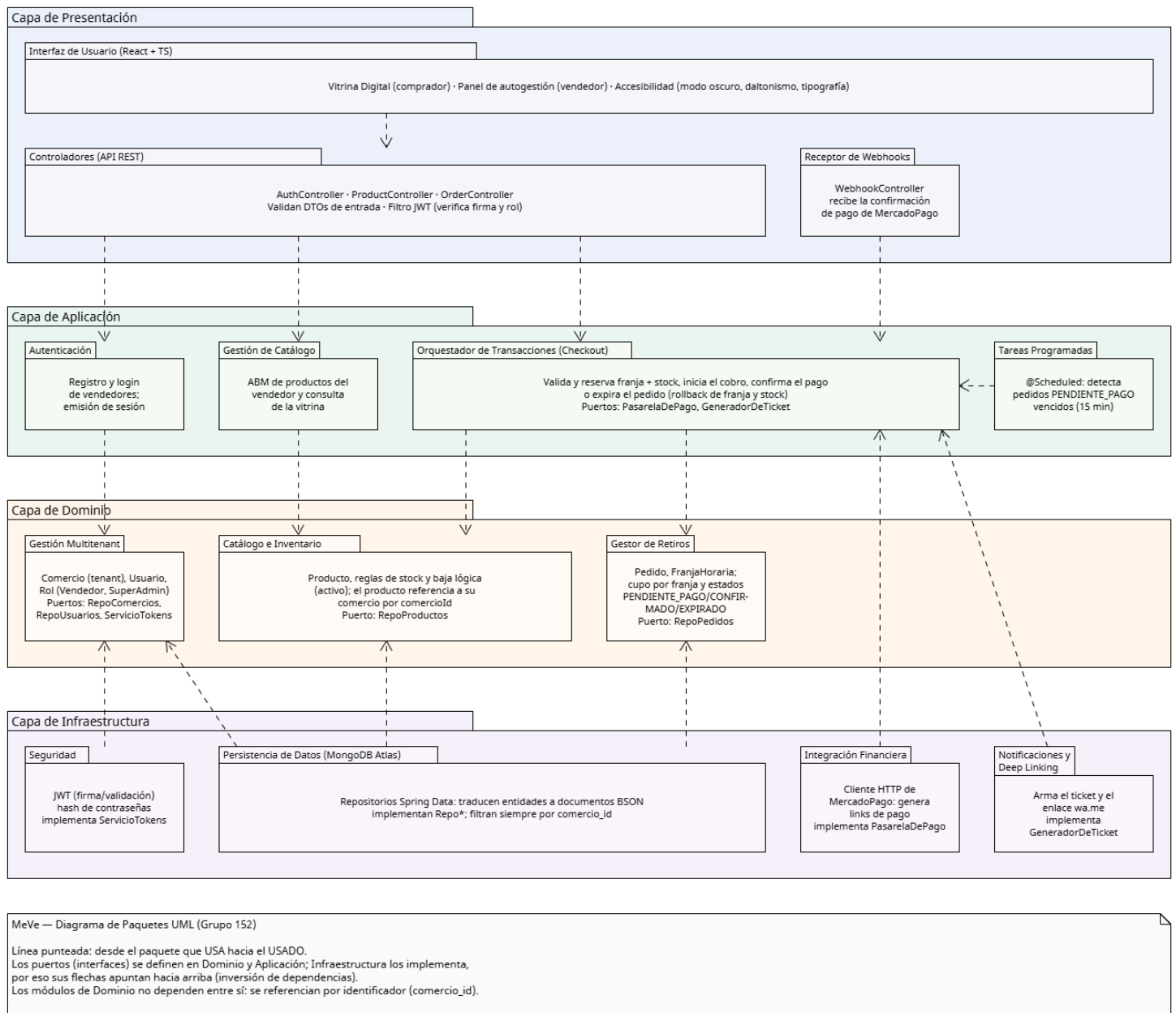
- **Módulo de Persistencia de Datos:** Contiene la implementación de los repositorios de Spring Data MongoDB. Se encarga de traducir las Entidades de negocio en Documentos NoSQL (BSON) para su guardado y recuperación en MongoDB Atlas, manejando la flexibilidad de esquemas que requieren los catálogos heterogéneos.

- **Módulo de Seguridad:** Se encarga de la firma y validación de los tokens JWT y del hash de las contraseñas. Implementa el puerto **ServicioTokens**.
- **Módulo de Integración Financiera (Pasarela de Pago):** Actúa como adaptador externo. Contiene los clientes HTTP responsables de comunicarse con las APIs de MercadoPago, enviar las credenciales del vendedor, generar los links de pago y exponer el receptor seguro para los webhooks de confirmación.
- **Módulo de Notificaciones y Deep Linking:** Servicio técnico encargado de formatear el objeto final de la orden de compra y generar la URL dinámica hacia la API de WhatsApp, produciendo el ticket estructurado que consolida la etapa *Offline* del O2O Commerce.



Diagrama de Paquetes UML

La Figura 1 Representa los módulos de Meve agrupados en cuatro capas. Las dependencias se indican con líneas punteadas que parten del paquete que usa y apunta al paquete usado.



Siguiendo la sugerencia de la cátedra, se utilizó asistencia de la IA para obtener un primer borrador de la arquitectura. Ese resultado no se aceptó tal cual: se revisó cada dependencia y se introdujeron los siguientes cambios.

- Se agregaron los módulos de autenticación y Gestión de Catálogo en la capa de Aplicación. Sin ellos AuthController y ProductController debían acceder directamente al Dominio, saltando una capa.
- El receptor de webhooks pasó de Infraestructura a Presentación, porque recibir peticiones HTTP es responsabilidad de la frontera del sistema. La infraestructura conserva únicamente las comunicaciones salientes.
- Se incorporó el Módulo de Seguridad en infraestructura, para aislar la gestión de token y contraseñas del resto del sistema.

- Las tareas Programadas pasaron a depender solo del Orquestador de Transacciones, en lugar de liberar franjas y stock por su cuenta, para que la lógica de compensación no quede duplicada.
- Se eliminaron las dependencias entre módulos de Dominio, que se relacionan por identificador, reduciendo en acoplamiento del núcleo del sistema.

Listado y Responsabilidad de los Módulos del Repositorio

El proyecto se aloja en un único repositorio de GitHub (github.com/taroutn/meve), organizado en las siguientes carpetas de primer nivel:

- backend/ : API REST desarrollada en Spring Boot, que contiene las capas de Presentación (lado servidor), Aplicación, Dominio e Infraestructura.
- frontend/ : cliente web desarrollado en React y TypeScript.
- docs/ : documentación del proyecto, incluida la colección de Postman con los contratos de la API(docs/api).
- .github/workflows/ : pipeline de integración continua que compila y ejecuta las pruebas de backend y del frontend en cada pull request. la conveccion de ramas del equipo se cocuementa en CONTRIBUTING.md

Dentro del backend, cada módulo lógico se corresponde con un paquete Java bajo com.meve.backend, organizado por capas. la siguiente tabla resume la ubicación y la responsabilidad de cada uno:

Módulo	Capa	Ubicación en el repositorio (propuesta)	Responsabilidad
Interfaz de Usuario	Presentación	frontend/src	Vitrina y panel del vendedor; consume la API REST.
Controladores	Presentación	presentacion.controladores	Endpoints REST, validación de DTOs y filtro JWT.
Receptor de Webhooks	Presentación	presentacion.webhooks	Recepción de notificaciones de pago de MercadoPago.
Autenticación	Aplicación	aplicacion.autenticacion	Registro, login y emisión de la sesión.
Gestión de Catálogo	Aplicación	aplicacion.catalogo	ABM de productos y consulta de la vitrina.
Orquestador de Transacciones	Aplicación	aplicacion.checkout	Reserva, cobro, confirmación y expiración del pedido.
Tareas Programadas	Aplicación	aplicacion.tareas	Detección periódica de pedidos vencidos.
Gestión Multitenant	Dominio	dominio.multitenant	Comercio, Usuario, Rol y reglas de pertenencia.
Catálogo e Inventario	Dominio	dominio.catalogo	Producto y reglas de stock.

Gestor de Retiros	Dominio	dominio.retiros	Pedido, franjas, cupos y estados.
Persistencia de Datos	Infraestructura	infraestructura.persistencia	Repositorios de Spring Data MongoDB.
Seguridad	Infraestructura	infraestructura.seguridad	Tokens JWT y hash de contraseñas.
Integración Financiera	Infraestructura	infraestructura.pagos	Cliente HTTP de MercadoPago.
Notificaciones y Deep Linking	Infraestructura	infraestructura.notificaciones	Ticket del pedido y enlace wa.me.



Diseño y Esquema de la Base de Datos

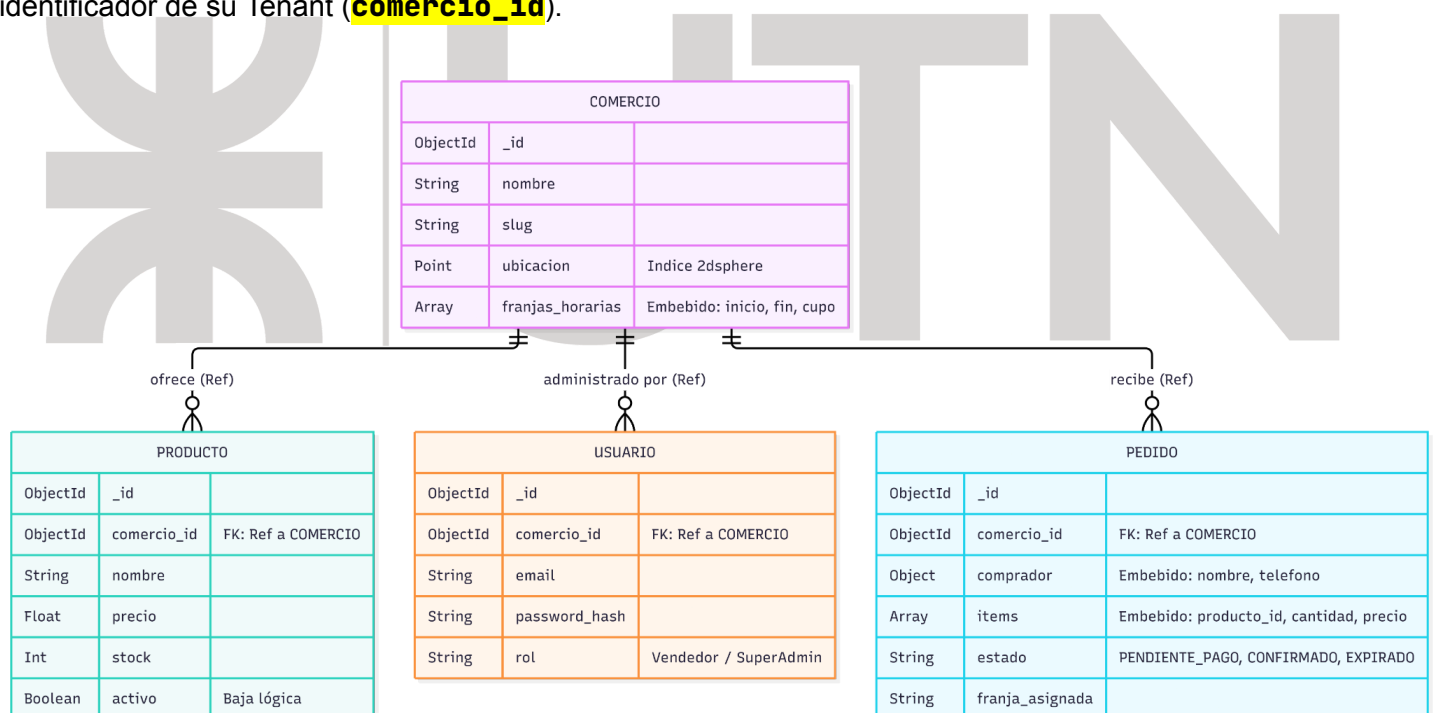
En esta sección se detalla el modelado de datos para la plataforma MeVe. A diferencia de un enfoque relacional tradicional que prioriza la pureza estructural y la normalización extrema, el diseño de esta base de datos se ha orientado hacia los patrones de acceso a los datos, es decir, a cómo la aplicación consultará, insertará y actualizará la información en tiempo real.

Justificación del Enfoque No Relacional (MongoDB)

La elección de MongoDB responde directamente a la naturaleza de los actores del sistema. Al tratarse de un marketplace hiperlocal que abarca desde viandas gastronómicas hasta manufacturas artesanales, los esquemas relacionales tradicionales resultan un obstáculo. La naturaleza NoSQL de MongoDB permite almacenar catálogos de productos completamente heterogéneos mediante documentos flexibles, adaptándose constantemente a la oferta dispar de cada micro-comerciante sin forzar estructuras rígidas.

Diagrama Conceptual y Estructura Documental

El diseño sigue el principio NoSQL donde las entidades conceptuales se transforman en colecciones, y cada instancia se representa como un documento (formato JSON/BSON). Para sostener la arquitectura multi-tenant, el esquema garantiza que los documentos operativos mantengan una referencia directa al identificador de su Tenant (**comercio_id**).



Estrategia de Almacenamiento: Referencias y Documentos Embebidos

En el contexto de bases de datos no relacionales, se acepta cierta desnormalización intencional (duplicación controlada de datos) como un *trade-off* para mejorar la eficiencia en escenarios de gran volumen y reducir la necesidad de operaciones de unión en tiempo de lectura.

Para MeVe, la estrategia se divide en dos enfoques:

- **Documentos Embebidos (Desnormalización):** Se privilegia el almacenamiento conjunto de datos que tienen una fuerte relación de pertenencia y que se consultan siempre en bloque. Por ejemplo, en la colección **PEDIDO**, los datos del comprador y el detalle de los ítems (con su precio al momento de la compra) se almacenan de forma embebida. Esto garantiza la inmutabilidad del ticket y la extrema rapidez de lectura. Del mismo modo, la configuración de *franjas horarias* se embebe dentro del documento de **COMERCIO**.
- **Referencias:** Se utilizan para vincular entidades independientes que crecen a ritmos distintos y necesitan ser consultadas por separado, como la relación entre un **PRODUCTO** y el **COMERCIO** al que pertenece.



Anexo

Porcentajes de participación de la entrega:

- Fabián: 33,3%
- Tiziano: 33,3%
- Lautaro: 33,3%

Suma total de carga de trabajo: 100%

